

Classes

Lox에 객체지향을 더하다 — 클래스 선언, 인스턴스, 필드, 메서드, `this`, 생성자

jlox · Java tree-walking interpreter

이 장에서 만드는 것

한 줄 한 줄이 동작하는 모습으로 정리

```
class Breakfast {  
    init(meat, bread) {          // 생성자  
        this.meat = meat;        // 필드 설정  
        this.bread = bread;  
    }  
  
    serve(who) {                 // 메서드  
        print "Enjoy your " + this.meat + ", " + who + ".";  
    }  
}  
  
var bacon = Breakfast("bacon", "toast"); // 클래스 호출 → 인스턴스  
bacon.serve("you");                     // Enjoy your bacon, you.
```

필요한 것

- 클래스 선언과 실행
- 클래스 호출 → 인스턴스
- 필드 읽기/쓰기 (.)

기능 하나가 인터프리터 전 단계를 가로지른다

스캐너는 그대로, 나머지 세 단계에 손을 댄다



새 AST 노드 4개 — `Stmt.Class`, `Expr.Get`, `Expr.Set`, `Expr.This` · 메서드는 기존 `Stmt.Function`을 그대로 재사용한다.

새 런타임 타입 2개 — `LoxClass`(클래스 자체), `LoxInstance`(인스턴스).

추가되는 문법

기존 문법에 얹히는 규칙들

```
// 선언
declaration → classDecl | funDecl | varDecl | statement ;
classDecl   → "class" IDENTIFIER "{" function* "}" ;

// 프로퍼티 접근 - 호출과 같은 우선순위 단계로 묶인다
call        → primary ( "(" arguments? ")" | "." IDENTIFIER )* ;

// 대입 - 좌변이 프로퍼티일 수 있다
assignment  → ( call "." )? IDENTIFIER "=" assignment | logic_or ;

// 기본식 - this
primary     → ... | "this" | IDENTIFIER ;
```

.(프로퍼티)은 함수 호출 ()과 같은 단계에서 왼쪽부터 체인으로 묶인다 — `egg.scramble(3).with(cheddar)`.

① 클래스 선언 — 파싱

```
// Parser.java — 모든 선언의 진입점
private Stmt declaration() {
    try {
        if (match(CLASS)) return classDeclaration(); // ← 추가
        if (match(FUN))   return function("function");
        if (match(VAR))   return varDeclaration();
        return statement();
    } catch (ParseError e) { synchronize(); return null; }
}

private Stmt classDeclaration() {
    Token name = consume(IDENTIFIER, "Expect class name.");
    consume(LEFT_BRACE, "Expect '{' before class body.");

    List<Stmt.Function> methods = new ArrayList<>();
    while (!check(RIGHT_BRACE) && !isAtEnd()) {
        methods.add(function("method")); // 함수 파서를 그대로 재사용
    }
    consume(RIGHT_BRACE, "Expect '}' after class body.");
    return new Stmt.Class(name, methods);
}
```

메서드는 **fun 키워드 없는 함수**일 뿐 — `function()`을 재사용해 이름·파라미터·본문을 파싱한다.

① 클래스 선언 — 실행

```
// Interpreter.java - class 선언문을 실행
@Override
public Void visitClassStmt(Stmt.Class stmt) {
    environment.define(stmt.name.lexeme, null);           // ① 이름만 먼저 선언

    Map<String, LoxFunction> methods = new HashMap<>();
    for (Stmt.Function method : stmt.methods) {
        LoxFunction function = new LoxFunction(
            method, environment,
            method.name.lexeme.equals("init"));           // init이면 생성자 표시
        methods.put(method.name.lexeme, function);
    }

    LoxClass clazz = new LoxClass(stmt.name.lexeme, methods);
    environment.assign(stmt.name, clazz);                // ② 진짜 클래스로 채움
    return null;
}
```

왜 define → assign 2단계인가? 먼저 이름을 null로 선언해두면, 메서드 본문 안에서 자기 클래스 이름을 참조할 수 있다.

② 인스턴스 생성 — 클래스를 호출한다

클래스가 곧 LoxCallable — 호출하면 인스턴스를 만든다

```
// LoxClass.java
class LoxClass implements LoxCallable {
    final String name;
    private final Map<String, LoxFunction> methods;

    @Override
    public Object call(Interpreter interpreter, List<Object> arguments) {
        LoxInstance instance = new LoxInstance(this); // 빈 인스턴스
        return instance;
    }

    LoxFunction findMethod(String name) {
        return methods.get(name); // 없으면 null
    }
}
```

Breakfast("bacon", "toast") → 기존 visitCallExpr이 LoxCallable.call()을 부른다. 함수 호출과 완전히 같은 길.

② 인스턴스의 정체

인스턴스는 **클래스 참조 + 필드 맵** — 그게 전부다

```
// LoxInstance.java
class LoxInstance {
    private LoxClass clazz;
    private final Map<String, Object> fields = new HashMap<>();

    LoxInstance(LoxClass clazz) { this.clazz = clazz; }

    @Override
    public String toString() { return clazz.name + " instance"; }
}
```

필드(field)

인스턴스마다 가지는 **상태**. 자유롭게 추가 가능 — 선언 불필요.

메서드(method)

클래스가 소유하고 모든 인스턴스가 공유. 필드 맵엔 없다.

③ 프로퍼티 접근 — 파싱

읽기: **a.b** → Get

```
// Parser.call()
while (true) {
    if (match(LEFT_PAREN)) {          // 호출
        expr = finishCall(expr);
    } else if (match(DOT)) {          // ← 추가
        Token name = consume(IDENTIFIER,
            "Expect property name after '.'.");
        expr = new Expr.Get(expr, name);
    } else break;
}
```

쓰기: **a.b = c** → Set

```
// Parser.assignment()
Expr expr = or();                    // 좌변: a.b → Get
if (match(EQUAL)) {
    Expr value = assignment();
    if (expr instanceof Expr.Variable) {
        ...                          // 변수 대입
    } else if (expr instanceof Expr.Get) {
        Expr.Get get = (Expr.Get) expr;
        return new Expr.Set(
```

③ 프로퍼티 접근 — 실행

get — 필드 먼저, 그다음 메서드

```
// LoxInstance.get()
Object get(Token name) {
    if (fields.containsKey(name.lexeme))    // ①필드
        return fields.get(name.lexeme);

    LoxFunction method =                  // ②메서드
        clazz.findMethod(name.lexeme);
    if (method != null)
        return method.bind(this);        // this 바인딩!

    throw new RuntimeError(name,
        "Undefined property '" + name.lexeme + "'");
}
```

set — 필드에 그냥 넣는다

```
// Interpreter.visitSetExpr()
Object object = evaluate(expr.object);
if (!(object instanceof LoxInstance))
    throw new RuntimeError(expr.name,
        "Only instances have fields.");

Object value = evaluate(expr.value);
```

④ this — 메서드가 인스턴스를 기억하는 법

get에서 본 `method.bind(this)`가 핵심

```
// LoxFunction.bind()
LoxFunction bind(LoxInstance instance) {
  Environment env =
    new Environment(closure);
  env.define("this", instance); // 숨은 변수
  return new LoxFunction(
    declaration, env, isInitializer);
}
```

this는 메서드를 인스턴스에 묶을 때 생기는 숨은 지역변수다.

메서드 본문이 보는 환경 체인:

this 환경 this → instance

↓ enclosing

클래스 정의 환경 (closure)

↓ enclosing

전역 환경

④ this — Resolver가 스코프를 씌운다

메서드를 풀기 전에 this 한 칸짜리 스코프를 끼워 넣는다

```
// Resolver.visitClassStmt()
public Void visitClassStmt(Stmt.Class stmt) {
    ClassType enclosing = currentClass;
    currentClass = ClassType.CLASS;           // 지금 클래스 안이다

    declare(stmt.name); define(stmt.name);

    beginScope();
    scopes.peek().put("this", true);         // ← this 스코프 한 칸

    for (Stmt.Function method : stmt.methods) {
        FunctionType decl = method.name.lexeme.equals("init")
            ? FunctionType.INITIALIZER : FunctionType.METHOD;
        resolveFunction(method, decl);
    }

    endScope();
    currentClass = enclosing;
    return null;
}
```

덕분에 visitThisExpr은 this를 **평범한 지역변수처럼** resolve해, 정확한 거리(distance)를 기록한다.

⑤ 생성자 init

클래스를 호출하면 자동으로 불리는 특별한 메서드

```
// LoxClass.call() - init 자동 호출
LoxInstance instance =
    new LoxInstance(this);
LoxFunction init = findMethod("init");
if (init != null)
    init.bind(instance)
        .call(interpreter, arguments);
return instance;
```

호출의 arity도 init의 파라미터 수를 따른다.

```
// LoxFunction.call() - init은 늘 this 반환
try {
    interpreter.executeBlock(
        declaration.body, env);
} catch (Return r) {
    if (isInitializer)
        return closure.getAt(0, "this");
    return r.value;
}
if (isInitializer)
    return closure.getAt(0, "this");
return null;
```

Resolver가 런타임 전에 잡아주는 것

this — 클래스 밖 금지

```
// visitThisExpr
if (currentClass == ClassType.NONE) {
    Lox.error(expr.keyword,
        "Can't use 'this'
        outside of a class.");
    return null;
}
resolveLocal(expr, expr.keyword);
```

init — 값 반환 금지

```
// visitReturnStmt
if (stmt.value != null) {
    if (currentFunction
        == FunctionType.INITIALIZER)
        Lox.error(stmt.keyword,
            "Can't return a value
            from an initializer.");
    resolve(stmt.value);
}
```

정리

추가된 것

새 AST

Stmt.Class

Expr.Get

Expr.Set

Expr.This

새 런타임 타입

- LoxClass — Callable + 메서드 보관 + 인스턴스 팩토리
- LoxInstance — 클래스 참조 + 필드 맵

핵심 통찰

- 클래스 호출 = 인스턴스 생성 (Callable 재사용)
- 메서드 = fun 없는 함수 (파서 재사용)
- Get으로 파싱 → = 보면 Set으로 재해석
- 필드 > 메서드 우선순위(shadowing)
- `this = bind`가 만드는 환경 변수
- `init` = 항상 `this`를 돌려주는 메서드